

SnapVM: Snapshot-Based MicroVM Environments for Fast Execution and Recovery

Arthur Carvalho Rodrigues  [Pontifícia Universidade Católica de Minas Gerais | arthurcarvalhorodrigues2409@gmail.com]

Débora Luiza de Paula Silva  [Pontifícia Universidade Católica de Minas Gerais | debora.paula05@gmail.co]

Gustavo Henrique Rodrigues de Castro  [Pontifícia Universidade Católica de Minas Gerais | guguh1612@gmail.com]

Mariana Almeida Mendonça  [Pontifícia Universidade Católica de Minas Gerais | marianaalmeidaafa@gmail.com]

✉ Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, R. Dom José Gaspar, 500 – Coração Eucarístico, Belo Horizonte – MG, 30535-901, Brazil.

Resumo. Este trabalho apresenta o SnapVM, um sistema experimental de orquestração de microVMs construído sobre o Firecracker da AWS. O projeto investiga o uso de snapshots como primitiva central de gerenciamento de estado em ambientes de execução virtualizados, permitindo inicialização instantânea, rollback determinístico e clonagem de ambientes a partir de um estado capturado. O artigo descreve a motivação técnica do projeto, revisa a literatura relevante nas áreas de virtualização leve, mecanismos de checkpoint e execução em sandbox, e apresenta a metodologia de desenvolvimento adotada.

Keywords: MicroVM; Firecracker; Snapshots; Virtualização Leve; Execução Isolada; Serverless;

© Published under the Creative Commons Attribution 4.0 International Public License (CC BY 4.0)

1 Introdução

A computação em nuvem moderna, especialmente no modelo serverless, coloca uma demanda singular sobre a infraestrutura: executar cargas de trabalho de múltiplos clientes no mesmo hardware físico, com overhead mínimo, ao mesmo tempo em que garante isolamento robusto de segurança e desempenho. Agache *et al.* [2020], ao descreverem a motivação para o desenvolvimento do Firecracker na AWS (Amazon Web Services), sintetizaram esse dilema como a visão tradicional de que existe uma escolha entre virtualização com forte segurança e alto overhead, e tecnologias de container com segurança mais fraca e overhead mínimo. Para provedores de infraestrutura pública, esse tradeoff é inaceitável: ambos os requisitos — segurança e eficiência — são inegociáveis.

Containers, como o Docker, resolvem parte do problema ao usar namespaces e cgroups do Linux para isolar processos. Porém, conforme analisado por Jain [2020] e confirmado por Weissman *et al.* [2023], containers compartilham o kernel do sistema operacional hospedeiro, o que significa que uma vulnerabilidade no kernel pode comprometer todos os containers em execução no mesmo host. Essa dependência de um kernel compartilhado é especialmente problemática em cenários que envolvem código não confiável, agentes autônomos de IA ou experimentos reproduzíveis que exigem garantias mais fortes de isolamento do que um limite de processo pode oferecer.

Máquinas virtuais tradicionais, baseadas em hipervisores como o QEMU/KVM, oferecem isolamento por hardware — cada VM executa seu próprio kernel, em espaço de memória separado. No entanto, o custo dessa segurança é alto: o conjunto QEMU/KVM soma mais de 1 milhão de linhas

de código e requer até 270 chamadas de sistema distintas [Agache *et al.*, 2020], tornando o tempo de inicialização e o overhead de memória proibitivos para ambientes que precisam provisionar centenas ou milhares de instâncias rapidamente. A computação serverless, em particular, não tolera latências de boot na ordem de segundos.

O Firecracker surgiu como uma resposta direta a esse dilema. Desenvolvido pela AWS e apresentado no NSDI 2020 [Agache *et al.*, 2020], ele é um Virtual Machine Monitor (VMM) escrito em Rust que utiliza o KVM do Linux para executar microVMs com footprint de memória inferior a 5 MB por instância e tempo de boot inferior a 125 ms. Ao eliminar deliberadamente emulação de dispositivos legados — USB, BIOS, PCI — e ao expor uma API REST minimalista via socket local, o Firecracker desloca a fronteira crítica de segurança do limite do sistema operacional para um limite suportado por hardware. Desde 2018, ele sustenta em produção os serviços AWS Lambda e Fargate, processando trilhões de requisições por mês [Jain, 2020].

Ainda assim, o Firecracker é uma peça de infraestrutura de baixo nível — ele não resolve, por si só, a gestão de ciclo de vida de ambientes de execução. Criar um snapshot, restaurá-lo, bifurcar múltiplas instâncias a partir de um estado capturado: essas operações exigem orquestração que o Firecracker não oferece nativamente. É exatamente essa lacuna que o SnapVM se propõe a preencher. O projeto não replica o Firecracker; ele constrói sobre ele uma camada de mais alto nível que trata snapshots como a primitiva central de gerenciamento de estado — transformando microVMs em ambientes com capacidade de retrocesso no tempo, clonagem e recuperação instantânea após falhas.

1.1 Motivação

- Sandboxes baseados em containers (Docker, Podman) compartilham o kernel do host, limitando o isolamento real para cargas de trabalho não confiáveis [Weissman *et al.*, 2023].
- Máquinas virtuais completas (QEMU, VirtualBox) oferecem isolamento robusto, mas sofrem com tempos de inicialização lentos e alto consumo de memória [Agache *et al.*, 2020].
- Mecanismos de snapshot existem em hipervisores, mas raramente são expostos como primitivas de ciclo de vida de primeira classe acessíveis via CLI ou API programática.
- Execução de agentes de IA, testes automatizados e fluxos de pesquisa reproduzível exigem re-execução determinística a partir de um estado conhecido — funcionalidade ausente na maioria das ferramentas [Long *et al.*, 2020].

1.2 Objetivos

1. Projetar e implementar uma ferramenta CLI que gerencie microVMs Firecracker ao longo de seu ciclo de vida completo.
2. Expor a criação e restauração de snapshots como operações primárias, não como detalhes de implementação.
3. Permitir a clonagem de ambientes de VM a partir de um snapshot base compartilhado com overhead próximo de zero.
4. Validar a abordagem como base prática para execução de código em sandbox, frameworks de agentes e testes reproduzíveis.
5. Estabelecer métricas de desempenho de referência para latência de criação de snapshots, tempo de restauração e consumo de memória.

1.3 Escopo e Limitações

O SnapVM é um sistema experimental voltado para ambientes host baseados em Linux em hardware x86_64 com suporte a KVM. O projeto não tem como objetivo substituir hipervisores de produção ou runtimes de containers de propósito geral. Redes, armazenamento persistente e distribuição multi-host estão intencionalmente fora do escopo da fase atual.

2 Revisão Bibliográfica

O design do SnapVM se apoia em múltiplas áreas de pesquisa que se intersectam: virtualização leve, mecanismos de snapshot e checkpoint, execução de código em sandbox e ambientes para agentes de IA. Esta seção revisa os trabalhos fundamentais e os sistemas contemporâneos que informam a arquitetura e as decisões de projeto do SnapVM.

2.1 Virtualização Leve

O Firecracker [Agache *et al.*, 2020] é a tecnologia fundamental subjacente ao SnapVM. Desenvolvido pela AWS para al-

imentar o Lambda e o Fargate, é um VMM construído em Rust que usa o subsistema KVM do Linux para rodar microVMs com overhead mínimo. Conforme detalhado por Jain [2020], o Firecracker inicializa um kernel guest em menos de 125 ms, suporta configurações de memória a partir de 128 MB e expõe uma API REST para gerenciamento de VMs via socket local. Sua arquitetura omite deliberadamente emulação de dispositivos (USB, BIOS, PCI) para reduzir a superfície de ataque e aumentar a densidade de instâncias por host.

O gVisor [Young *et al.*, 2019] oferece uma abordagem alternativa de isolamento — interceptando chamadas de sistema via um kernel em espaço de usuário. Embora o gVisor alcance isolamento robusto sem exigir virtualização de hardware, seu overhead em cargas de trabalho intensivas em syscalls é mensurável: estudos apontam que o gVisor é até 2x mais lento que o Docker nativo em determinados workloads [Long *et al.*, 2020]. A escolha do SnapVM pela virtualização assistida por hardware via KVM favorece isolamento mais forte e desempenho mais previsível.

2.2 Mecanismos de Snapshot e Checkpoint

Mecanismos de snapshot de VM foram amplamente estudados no contexto de migração ao vivo e tolerância a falhas. O CRIU (*Checkpoint/Restore In Userspace*) permite o checkpointing de processos Linux sem suporte a virtualização de hardware, capturando descritores de arquivo, mapas de memória e estado da CPU [CRIU Project, 2012]. Embora poderoso, o CRIU opera na granularidade de processos e não oferece o isolamento completo de sistema que uma fronteira de microVM proporciona.

O Kata Containers [Kata Containers Contributors, 2017] combina semântica de containers com isolamento de nível de VM, usando QEMU ou Firecracker como VMM subjacente. O projeto demonstra que containers com suporte de VM são operacionalmente viáveis, mas prioriza compatibilidade OCI em detrimento de um design centrado em snapshots. O SnapVM diverge aqui ao tratar snapshots como a interface primária, não como detalhe de implementação.

Pesquisas sobre bifurcação de VMs no nível do hipervisor [Lagar-Cavilla *et al.*, 2009] demonstraram que é possível instanciar múltiplas VMs filhas a partir de um estado pai por meio de páginas de memória com cópia-na-escrita (*copy-on-write*). O modelo de clonagem do SnapVM se inspira conceitualmente nesse trabalho, embora implementado em uma camada de orquestração de nível mais alto, sem modificações no hipervisor.

2.3 Execução de Código em Sandbox

Plataformas de execução de código como Judge0, Sphere Engine e Replit operacionalizaram a execução em sandbox em escala, empregando namespaces do Linux, cgroups e filtragem seccomp. Plataformas mais recentes — como o E2B [E2B, 2023] — adotaram o Firecracker como backend para oferecer isolamento mais robusto. O E2B é um dos sistemas contemporâneos mais próximos do design pretendido pelo SnapVM: expõe ambientes de VM como contextos de execução sob demanda voltados para desenvolvedores. O

SnapVM se diferencia ao focar em orquestração local, auto-hospedada e nativa de snapshots, sem depender de infraestrutura de nuvem gerenciada.

2.4 Execução Reproduzível e Determinística

A reprodutibilidade em ambientes computacionais é abordada por ferramentas como Nix, Guix e Docker BuildKit — cada uma garantindo que uma especificação de ambiente produza resultados idênticos entre builds. Esses sistemas alcançam reprodutibilidade por meio de determinismo de build. O SnapVM propõe uma abordagem complementar: em vez de re-derivar um ambiente a partir de uma especificação a cada execução, captura e restaura diretamente um estado completamente inicializado e configurado. As duas abordagens são ortogonais e podem coexistir.

2.5 Ambientes de Execução para Agentes de IA

À medida que agentes de IA executam código, navegam na web e interagem com sistemas de arquivos, a necessidade de ambientes de execução seguros e reversíveis tornou-se um desafio de engenharia urgente. Frameworks como o interpretador de código da OpenAI, o uso de computador da Anthropic e frameworks de código aberto (AutoGPT, LangGraph) exigem sandboxes que possam ser iniciados rapidamente, isolados do host e revertidos após cada ação. O modelo de snapshot-restore do SnapVM é diretamente aplicável a esse padrão, permitindo que agentes operem em estados de VM em tempo real com a capacidade de retroceder após qualquer etapa — sem necessidade de reconstruir o ambiente do zero.

3 Metodologia

A metodologia do SnapVM segue um ciclo de desenvolvimento iterativo e orientado a experimentos. O projeto é estruturado em fases distintas, cada uma validando uma capacidade central antes que a próxima comece. Esta seção descreve a abordagem técnica, as decisões de stack tecnológico e a estratégia de validação adotadas.

3.1 Design da Pesquisa

O desenvolvimento do SnapVM adota um design de pesquisa com prototipagem prioritária (*prototyping-first*). Em vez de especificar uma arquitetura completa do sistema antecipadamente, os comportamentos centrais são implementados incrementalmente e validados contra critérios mensuráveis. Cada capacidade — inicialização de VM, criação de snapshot, restauração de snapshot, execução de múltiplos clones — é tratada como um experimento independente com entradas definidas, saídas esperadas e limiares de aceitação.

Essa escolha metodológica é motivada pela natureza exploratória do projeto: as características de desempenho do Firecracker em diferentes configurações de workload não são totalmente previsíveis a priori, e as primitivas de orquestração mais adequadas emergem da experimentação prática.

3.2 Stack Tecnológico

As decisões tecnológicas que fundamentam a implementação do SnapVM estão descritas na Tabela 1.

Table 1. Stack tecnológico do SnapVM

Componente	Tecnologia
VMM	Firecracker v1.x
SO Host	Linux (Ubuntu 22.04+)
Linguagem CLI	Go ou Rust (a definir)
Kernel Guest	Linux 5.10 (config microvm)
Rootfs Guest	Alpine Linux (imagem ext4)
IPC / API	REST Firecracker (Unix socket)
Armazenamento de Snapshots	Sistema de arquivos local (ext4/xfs)

3.3 Fases de Desenvolvimento

O desenvolvimento do SnapVM é organizado em quatro sprints sequenciais, cada um com escopo e entregáveis bem definidos.

3.3.1 Sprint 1 — Pesquisa e Validação de Conceitos

O Sprint 1 é dedicado à pesquisa do Firecracker, validação de conceitos fundamentais e realização de pequenos experimentos com snapshots. Os objetivos incluem:

- Estudar a arquitetura interna do Firecracker e seu modelo de API REST [Jain, 2020].
- Inicializar manualmente uma microVM e verificar os parâmetros de boot (tempo, memória, isolamento).
- Executar o ciclo básico de snapshot: pausar a VM, capturar estado (memória + disco) e restaurar.
- Documentar os resultados como baseline para os sprints seguintes.

3.3.2 Sprint 2 — Implementação da CLI

O Sprint 2 foca na implementação do SnapVM como uma ferramenta de linha de comando (CLI), abstraindo a API REST do Firecracker em comandos de alto nível. Os entregáveis são comandos reais para gerenciar o ciclo de vida das VMs:

- `snapvm vm start` — inicializa uma nova VM a partir de um par kernel + rootfs.
- `snapvm vm stop` — encerra uma VM em execução de forma controlada.
- `snapvm snapshot create <nome>` — pausa e captura o estado completo de uma VM.
- `snapvm snapshot restore <nome>` — restaura uma VM a partir de um snapshot nomeado.

3.3.3 Sprint 3 — Refinamento e Validação do Sistema

O Sprint 3 tem como objetivo refinar a CLI, executar testes de validação do sistema e finalizar a documentação técnica. As atividades previstas incluem:

- Testes de regressão sobre o ciclo completo: boot, snapshot, restore e rollback.

- Medição sistemática das métricas de desempenho definidas nos critérios de avaliação (Seção 3.4).
- Tratamento de casos de erro: falhas de restore, snapshots corrompidos e timeouts de boot.
- Redação e revisão da documentação de uso da CLI e da arquitetura do sistema.

3.3.4 Sprint 4 — Integração com Agentes de IA e Execução Paralela

O Sprint 4 expande o SnapVM com suporte direto a agentes de IA e execução de múltiplas VMs em paralelo. Esse sprint endereça os casos de uso mais avançados do projeto:

- `snapvm vm clone <snapshot>` — lança N instâncias a partir do mesmo snapshot base.
- Rollback automático em códigos de saída não-zero de comandos executados no guest.
- SDK programático (Go/Python) para que frameworks de agentes controlem o ciclo de vida de VMs diretamente.
- Execução paralela de clones com agregação de resultados e isolamento garantido entre instâncias.

3.4 Critérios de Avaliação

O sucesso será avaliado contra os seguintes critérios mensuráveis:

- Tempo de boot frio da VM inferior a 500 ms no hardware de referência.
- Tempo de criação de snapshot inferior a 200 ms para VMs com até 256 MB de memória guest.
- Tempo de restauração de snapshot inferior a 150 ms (excluindo recarga de memória do disco).
- Fidelidade de estado: 100% do estado em memória e descritores de arquivo abertos preservados em ciclos de snapshot-restore.
- Overhead de inicialização de clone inferior a 50 ms por instância em relação ao baseline de restauração.

3.5 Riscos e Mitigações

- **Disponibilidade de KVM em hardware CI/CD:** mitigado direcionando para ambientes bare-metal ou com virtualização aninhada habilitada; modo de fallback com QEMU para desenvolvimento local.
- **Crescimento do tamanho de snapshots de memória:** mitigado avaliando formatos de snapshot comprimidos e snapshots delta em relação a uma imagem base.
- **Estado de rede após restaurações:** o estado do dispositivo de rede é intencionalmente excluído do escopo das Fases 1–2; checkpoint completo de rede é adiado para a Fase 3 ou posterior.

4 Disponibilidade de Código / Repositório

O código-fonte, a documentação e os experimentos iniciais do projeto SnapVM estão disponíveis publica-

mente no repositório oficial da disciplina no GitHub Classroom, que pode ser acessado através do seguinte link: <https://github.com/ICEI-PUC-Minas-CC-TI/plmg-cc-ti5-2026-1-snapvm/tree/main>.

References

- Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association.
- CRIU Project (2012). CRIU: Checkpoint/restore in userspace. <https://criu.org>. Acesso em: março 2026.
- E2B (2023). E2B: Cloud runtime for AI agents. <https://e2b.dev>. Acesso em: março 2026.
- Jain, M. (2020). Study of Firecracker MicroVM. *arXiv preprint arXiv:2005.12821*.
- Kata Containers Contributors (2017). Kata containers: The speed of containers, the security of VMs. <https://katacontainers.io>. Acesso em: março 2026.
- Lagar-Cavilla, H. A., Whitney, J. A., Scannell, A., Patchin, P., Rumble, S. M., de Lara, E., Brudno, M., and Satyanarayanan, M. (2009). SnowFlock: Rapid virtual machine cloning for cloud computing. In *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys '09)*, pages 1–12. ACM.
- Long, J., Tai, H.-Y., Hsieh, S.-T., and Yuan, M. J. (2020). A lightweight design for serverless Function-as-a-Service. *arXiv preprint arXiv:2010.07115*.
- Weissman, Z., Tiemann, T., Eisenbarth, T., and Sunar, B. (2023). Microarchitectural security of AWS Firecracker VMM for serverless cloud platforms. *arXiv preprint arXiv:2311.15999*.
- Young, E. G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A. C., and Arpaci-Dusseau, R. H. (2019). The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*. USENIX Association.